SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

SIMATS ENGINEERING
Engineer to Excel

DATA STRUCTURES

ASSIGNMENT 2

Name	D. Hasini
Register No.	192425147
Course Code	CSA0304
Course	Data Structures
Date	31 October 2026

1. Convert A * (B + C) to postfix notation manually. Show detailed steps and final result.

Introduction

In **infix notation** the operator is written between its operands, for example A + B. In **postfix notation** (also called Reverse Polish Notation) the operator is written after its operands, for example A B +. Postfix needs no brackets and no precedence rules at the time of evaluation, so a computer can evaluate it easily using a stack.

Given expression

A * (B + C)

Operator precedence used

Operator	Precedence	Associativity
( )	Highest (evaluated first)	-
* /	Higher	Left to right
+ -	Lower	Left to right

Method 1: Using an operator stack

Scan the expression from left to right and follow these rules:

1. If the symbol is an **operand** (A, B, C), write it directly to the output.
2. If the symbol is an **opening bracket "("**, push it on the stack.
3. If the symbol is an **operator**, pop operators from the stack to the output while the top has higher or equal precedence (the top is never popped for "("), then push the new operator.
4. If the symbol is a **closing bracket ")"**, pop operators to the output until "(" is found, then remove the "(".
5. At the end, pop all remaining operators from the stack to the output.

Step-by-step conversion

Step	Symbol read	Action	Stack (bottom to top)	Output
1	A	Operand, send to output	(empty)	A
2	*	Stack is empty, push *	*	A
3	(	Push (	* (	A
4	B	Operand, send to output	* (	A B
5	+	Top is "(", so push +	* (+	A B
6	C	Operand, send to output	* (+	A B C
7	)	Pop + to output until "(" is found; remove "("	*	A B C +
8	End	Pop remaining operator *	(empty)	A B C + *

Method 2: Quick manual check (bracket method)

1. Solve the bracket first: (B + C) becomes **B C +**.
2. The expression now looks like A * [B C +].
3. Place the operator * after both of its operands, A and [B C +]: **A B C + ***.

Final result: A B C + *

Verification

Take A = 2, B = 3, C = 4. Infix: 2 * (3 + 4) = 2 * 7 = 14.

Postfix evaluation of 2 3 4 + *: push 2, 3, 4; on + pop 4 and 3 and push 7; on * pop 7 and 2 and push 14. The answer is 14 in both cases, so the conversion is correct.

Note: the answer is not A B * C + because the brackets force B + C to be done before the multiplication.

2. Trace binary search to find the first occurrence of 20 in [10, 15, 20, 20, 25, 30]. Modify the algorithm to support this.

Introduction

Binary search finds a key in a **sorted** array. It compares the key with the middle element and discards the half that cannot contain the key, so the search range becomes half after every comparison. The time taken is O(log n), which is much faster than linear search, which takes O(n).

Given data

Index	**0**	**1**	**2**	**3**	**4**	**5**
Value	10	15	20	20	25	30

Key = 20. The key appears two times (index 2 and index 3), and we need the **first** (leftmost) occurrence. The middle index is calculated as mid = low + (high - low) / 2 using integer division.

Problem with the normal binary search

The normal algorithm stops as soon as it finds a match. When the array has duplicate values, the match it finds may not be the first one. For example, in [10, 20, 20, 20, 30] the first mid is index 2, so the normal algorithm returns 2, although the first 20 is at index 1.

Modified algorithm (first occurrence)

The idea: when the key is found at mid, **do not stop**. Save mid as a possible answer and continue searching in the **left half**, because an earlier occurrence can only be on the left.

1. Set low = 0, high = n - 1 and result = -1.
2. Repeat while low <= high:
    a) Find mid = low + (high - low) / 2.
    b) If A[mid] = key: store result = mid and set high = mid - 1 (search left for an earlier occurrence).
    c) Else if A[mid] < key: set low = mid + 1 (search the right half).
    d) Else (A[mid] > key): set high = mid - 1 (search the left half).
3. When the loop ends, return result. If result is still -1, the key is not present.

Trace of the modified algorithm

Initially low = 0, high = 5, result = -1, key = 20.

Step	low	high	mid	A[mid]	Check	Action	result
1	0	5	2	20	20 = 20	Match found. Save result = 2 and search left: high = 1	2
2	0	1	0	10	10 < 20	Go right: low = 1	2
3	1	1	1	15	15 < 20	Go right: low = 2	2
4	2	1	-	-	low > high	Loop ends	2

Final result: First occurrence of 20 is at index 2 (3rd position).

The second 20 at index 3 is on the right side, so it is never needed. The search checked only 3 elements out of 6.

Why the modification works

- Whenever a match is found, result keeps that index, and the search moves left to look for a smaller index.
- If no smaller index exists on the left, result is not changed and it remains the first occurrence.
- Since every match found later is further to the left, the last value stored in result is the leftmost match.

Complexity

Measure	Modified binary search	Linear search for first occurrence
Time (worst / average)	O(log n)	O(n)
Space	O(1)	O(1)
Requirement	Array must be sorted	No sorting needed

Conclusion: With a small change (store the index and continue to the left after a match), binary search can find the first occurrence of a repeated key while still taking only O(log n) time.